

```
---
title: "Class 6 R: functions"
author: "David Majeed (PID:A17885958)"
format: html
editor: visual
---
```

Background

All functions in R have 3 things:

- a **name** (we choose)
- input **arguments** (one or more comma-separated inputs that go inside the brackets)
- the **body** (the lines of R code that do the work of the function)

First Function

Here we will create a function to add some numbers. Called ``add()``

Can be `***"required" or "optional"***`, the later will have fall-back default values?

Q.1A

```
```{r}
add<-function(x,y=1000){
 x+y
}
```

#add is the name, function() is the function we use to make new function, the (x,y=1000) equal the inputs we are looking for, the =1000 is the default option if the user doesn't input anything, {} is what the function actually does  
```

Here we have the name: add, the inputs: x and y and the body of code: x+y

Can we use it?

```
```{r}
add(10,100)
add(10)
```

#we are testing our function  
```

Q1B. Lets modify the add function

This allows there to be a default option in case the user forgot to add one of the inputs

```
```{r}
add<-function(x,y=0){
 sum(x)+sum(y)
}
#modifying it to have the body be using the sum() function, that way it can add vectors as seen
bellow
add(4,7)
add(c(4,7,10))
```
```

Q.1C Lets add more inputs

This allows the user to add as many terms as they would like by using ...

```

```{r}
add<-function(x,y=0, ...){
 sum(x)+sum(y)+sum(...)
}
#the ... allow us to have unlimited inputs, that way we avoid errors if the user doesn't input a
variable or if they have more inputs than we coded for
add(1,2,3,4,5,7,7,7)
add(4, 7)
add(c(4,7,10))
add(1, 2, 3, -4)
```

```

We can ****return**** value from a function (rather than the default last line) with ``return()``

This helps us go back and just return the line we want

```

```{r}
add<-function(x,y=0, ...){
 return(sum(x)+sum(y)+sum(...))
 cat("is it break time?")
}
#return() prevents the code from going further
add(2,2)
```

```

As we see, the text isn't printed

A generate_dna() function

We can use the ``sample()`` function

This allows us to randomly choose a value from a vector or set, the size allows us to choose how many we select

```

```{r}
sample(1:5, size=5)
```

```

Q.2A We can use this to make a random nucleotide sequence if we draw from "A", "C", "G" and "T"

```

```{r}
#sample(c("A","C","G","T"), 1, replace = TRUE)

generate_dna <- function(len){
 sample(c("A","C","G","T"), len, replace = TRUE)
}

```

#we created a new function named generate\_dna that takes len as the input, and in return it gives a sample out of the available letters, for len # and we can replace letters after choosing them

```

generate_dna(len=6)
```

```

Instead of using numbers 1:5, we can use nucleotides. The number of nucleotides is selected from len input, and we are able to use more than 4 nucleotides by repeating them when replace=T

Q.2B

```

```{r}
generate_dna <- function(len,single.element){
ans <- sample(c("A","C","G","T"), len, replace = TRUE)
 if (single.element){
 cat(sample(c("A","C","G","T"), len, replace = TRUE), sep = "")}
else {return(ans)}
}

```

#if and else allow us to code for if the user wants to do an option or do the other one  
#sep removes the text to make a single string

```
generate_dna(len=2, single.element=T)
```
```

Building off the previous function, now we use `if` and `else`. These allow us to control what happens when a certain condition happens such as if we want to have a single character element which then goes through `cat` to turn it into a string of characters we want

Q2.C

```
## Fasta Format
```

```
```{r}
##Format as FASTA

generate_dna <- function(len,single.element=T){
 cat(paste(">len", len, "\n", sep=""))
 ans <- sample(c("A","C","G","T"), len, replace = TRUE)
 if (single.element){
 cat(sample(c("A","C","G","T"), len, replace = TRUE), sep = "")}
 else {return(ans)}
 cat("\n")
}
```

#cat controls characters, such as pasting the format of ">len", then adding the number from length, then "\n" goes down a line

```
generate_dna(len=2, single.element=T)
```

```
...
```

Now putting it all together, the above code use `cat` and `paste` to now format it properly

Q.3

```
generate_protein() function
```

```
```{r}
generate_protein <- function(len,single.element=T){
  cat(paste(">len", len, "\n", sep=""))
  ans <- sample(c("A", "R", "N", "D", "C", "E", "Q", "G", "H", "I", "L", "K", "M", "F", "P", "S",
    "T", "W", "Y", "V"), len, replace = TRUE)
  if (single.element){
    cat(sample(c("A", "R", "N", "D", "C", "E", "Q", "G", "H", "I", "L", "K", "M", "F", "P", "S",
    "T", "W", "Y", "V"), len, replace = TRUE), sep = "")}
  else {return(ans)}

  cat("\n")
}
```

#same thing as before modified for protein

```
generate_protein(66,T)
```
```

Q.4

Generate random protein sequences of length 6 to 13

```
```{r}
```

```
for (len in 6:13) {
  cat( generate_protein(len, T))
  cat("\n")
}
```

```
# for function allows us to do the same function multiple times without repeating it 8 times
```
```

Q.5

## Are our sequences found in nature?

| Length (aa) | Best hit % identity | Best hit % coverage | Unique? (Y/N) |
|-------------|---------------------|---------------------|---------------|
| 6           | 100                 | 100                 | N             |
| 7           | 100                 | 100                 | N             |
| 8           | 100                 | 88                  | Y             |
| 9           | 88.89               | 100                 | Y             |
| 10          | 90.00               | 100                 | Y             |
| 11          | 88.89               | 82                  | Y             |
| 12          | 88.89               | 75                  | Y             |
| 13          | 88.89               | 69                  | Y             |

Q.4A

After length 7, the sequences became unique in nature as either identity or coverage began to drop from 100%

Q.4B

The greater you increase the number of amino acids in the chain, the less likely will they be able to line up perfectly with a known sequence. This comes from the fact that there are 20 different possibilities of amino acid, thus adding length to the chain now severely decreases the probability that the entire sequence will line up with a known sequence, this can be modeled by  $20^{-(\text{length of peptide})}$ . Additionally, the known protein universe itself is limited to what humans have been able to sequence, although it grows everyday, there are way more proteins that haven't been sequenced

Q.6

As we can see throughout the class, most people got unique sequences starting around 7-9 amino acids. Thus it would be best to put the limit there, at 8. Any shorter, the immune system might accidentally match sequences that are from the self and thus cause self damage